

House Price Prediction using Machine Learning

Diploma Project Report

Student: Malishka

Project Type: Supervised Machine Learning – Regression

Project Objective

The objective of this project is to build and compare machine learning regression models for predicting house prices from housing-related numerical features.

House Price Prediction using Machine Learning

Diploma Project

Student: Malishka

Project Type: Supervised Machine Learning – Regression

Project Objective

The objective of this project is to build and compare machine learning regression models for predicting house prices from housing-related numerical features.

Workflow

1. Import and understand the dataset
2. Perform data cleaning and validation
3. Conduct Exploratory Data Analysis (EDA)
4. Split data into training and testing sets
5. Train multiple regression models
6. Evaluate and compare model performance
7. Select the best-performing model
8. Demonstrate sample predictions
9. Present conclusions and limitations

1. Import Libraries

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

2. Load Dataset

Code

```
# Keep boston.csv in the same folder as this notebook.
df = pd.read_csv("boston.csv")

print("Dataset shape:", df.shape)
display(df.head())
```

3. Basic Dataset Inspection

Code

```

print("Column names:")
print(df.columns.tolist())

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
display(df.describe())

```

4. Data Quality Checks

Code

```

print("Missing values by column:")
display(df.isnull().sum())

print("Total duplicate rows:", df.duplicated().sum())

# Remove duplicate rows if any are present
df = df.drop_duplicates().reset_index(drop=True)

print("Shape after duplicate removal:", df.shape)

```

5. Exploratory Data Analysis

Code

```

# Histograms
df.hist(figsize=(14, 10), bins=25)
plt.suptitle("Distribution of Dataset Features", fontsize=16)
plt.tight_layout()
plt.show()

```

Code

```

# Correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(numeric_only=True), cmap="coolwarm", annot=False)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

```

6. Define Features and Target

Code

```

# The Boston Housing dataset commonly uses MEDV as the target column.
# If your CSV uses a different target name, change target_column below.

target_column = "MEDV"

if target_column not in df.columns:
    # Try common alternatives
    candidates = [c for c in df.columns if c.upper() in ["MEDV", "PRICE", "TARGET"]]
    if candidates:
        target_column = candidates[0]
    else:
        raise ValueError("Target column not found. Set target_column to the correct column name.")

X = df.drop(columns=[target_column])
y = df[target_column]

print("Target column:", target_column)
print("Number of features:", X.shape[1])
print("Number of observations:", X.shape[0])

```

7. Train-Test Split

Code

```

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.20, random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

```

8. Train Multiple Machine Learning Models

Code

```

models = {
"Linear Regression": LinearRegression(),
"Decision Tree": DecisionTreeRegressor(random_state=42, max_depth=6),
"Random Forest": RandomForestRegressor(
n_estimators=300, random_state=42, max_depth=None, n_jobs=-1
)
}

results = []
trained_models = {}

for name, model in models.items():
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

results.append({
"Model": name,
"R2 Score": r2_score(y_test, y_pred),
"MAE": mean_absolute_error(y_test, y_pred),
"MSE": mean_squared_error(y_test, y_pred),
"RMSE": np.sqrt(mean_squared_error(y_test, y_pred))
})

trained_models[name] = model

results_df = pd.DataFrame(results).sort_values("R2 Score", ascending=False).reset_index(drop=True)
display(results_df)

```

9. Model Comparison

Code

```

plt.figure(figsize=(9, 5))
plt.bar(results_df["Model"], results_df["R2 Score"])
plt.ylabel("R2 Score")
plt.xlabel("Model")
plt.title("Model Performance Comparison - R2 Score")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()

```

10. Actual vs Predicted Values for the Best Model

Code

```

best_model_name = results_df.loc[0, "Model"]
best_model = trained_models[best_model_name]
best_predictions = best_model.predict(X_test)

print("Best model:", best_model_name)

plt.figure(figsize=(7, 6))
plt.scatter(y_test, best_predictions, alpha=0.7)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title(f"Actual vs Predicted - {best_model_name}")
plt.tight_layout()
plt.show()

```

11. Feature Importance

Code

```
if hasattr(best_model, "feature_importances_"):
    importance = pd.Series(
        best_model.feature_importances_, index=X.columns
    ).sort_values(ascending=False)

    plt.figure(figsize=(10, 6))
    importance.head(10).sort_values().plot(kind="barh")
    plt.xlabel("Importance")
    plt.title(f"Top Features - {best_model_name}")
    plt.tight_layout()
    plt.show()

    display(importance.to_frame("Importance"))
else:
    print("Feature importance is not directly available for the selected model.")
```

12. Sample Predictions

Code

```
# Show a small sample of actual vs predicted values
prediction_comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": best_predictions
}).head(10)

display(prediction_comparison)
```

13. Conclusion

This project demonstrates how supervised machine learning can be used for house-price prediction.

Three regression algorithms were trained and evaluated:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor

The models were compared using **R², MAE, MSE, and RMSE**. The model with the highest R² score and lower error values is selected as the best model for this dataset.

The project also includes exploratory data analysis, model comparison, actual-vs-predicted visualization, and feature-importance analysis where supported.

Limitations

- Model performance depends on the quality and representativeness of the dataset.
- This project is an educational prediction model and should not be treated as a real-world property valuation system without additional data and validation.
- The dataset is relatively small compared with modern real-estate datasets.

Future Scope

- Hyperparameter tuning using GridSearchCV/RandomizedSearchCV
- Cross-validation
- More advanced models such as Gradient Boosting/XGBoost
- A simple web interface using Streamlit
- Deployment as a prediction application

Viva / Presentation Points

1. What is the objective?

To predict house prices using supervised machine learning regression algorithms.

2. Why regression?

Because the target is a continuous numerical value (house price).

3. Why compare multiple models?

Different algorithms learn different relationships in the data. Comparing them helps identify a model that performs better on unseen test data.

4. What is R^2 ?

R^2 indicates how much of the variation in the target is explained by the model. Higher is generally better.

5. What is RMSE?

RMSE measures the typical prediction error while giving more weight to larger errors. Lower is better.

6. Why Random Forest can perform well?

It combines many decision trees and can model nonlinear relationships and feature interactions effectively.

7. What is overfitting?

Overfitting occurs when a model learns the training data too closely and performs poorly on unseen data.